

Adversarial Machine Learning

Homework Assignment 4

The assignment is due by the end of the day on Tuesday, April 1.

Please note that the submitted solutions are expected to follow the examples specified below that are posted on Canvas. If your code does not follow the provided examples, marks will be deducted. Also, marks will not be assigned if the codes are generated by coding assistants. To repeat, it is okay if you write the code and afterward use help from coding assistants to debug the code: it is not okay to ask a coding assistant to generate the code and then copy-paste it into the notebook.

And to remind you again, the assignments and solutions from the offering of the AML course in the previous semesters can be found at these links: Spring 2023 ([link](#)), and Fall 2021 ([link](#)).

Objective:

- To get familiar with deep learning models for Natural Language Processing (NLP), and implement adversarial attacks on NLP classification models.
- To get familiar with finetuning Large Language Models (LLMs), and implement jailbreaking attacks on LLMs.

Part 1: NLP Attacks (50 marks)

For this task, we will use the [TextAttack](#) library, which provides implementations of multiple adversarial attacks on NLP models. For model training and evaluation, we will use the Hugging Face library that is built on top of PyTorch. The notebook NLP_attacks in the NLP Classification folder on Canvas provides example code for training the NLP model and implementing adversarial attacks. **It is expected that your solution follows the example provided in the NLP_attacks notebook.**

Dataset: We will use a dataset consisting of 47,392 spam and non-spam messages. A Data Loader notebook for loading the dataset is provided in the folder for this assignment.

Task 1: Step 1: Train a **MobileBERT** neural network model for classification of the messages into spam and non-spam (ham) messages **using the Hugging Face library**. MobileBERT is a deep neural network model with a Transformer network architecture, and it is a smaller version of the widely used BERT model for text classification.

Use the provided Part1_DataLoader notebook in the assignment folder to load the dataset.

Follow the code in the NLP_attacks notebook posted in the Codes folder on Canvas for training and evaluating the model.

Similar to the example code in the notebook, import the tokenizer for MobileBERT from the folder “google/mobilebert-uncased” in the Hugging Face hub. Apply the following tokenizer function to convert the train, test, and validation text datasets into sequences of numerical tokens.

```
def tokenize(rows):
    return tokenizer(rows['text'], padding="max_length",
                     truncation=True, max_length=128)
```

Import the pretrained MobileBERT model from the folder “google/mobilebert-uncased” in the Hugging Face hub. Ensure that the number of classes of the imported model corresponds to the task of spam classification.

Finetune the MobileBERT model for classification of the spam and non-spam messages in the dataset. Finetuning with a small learning rate for a few epochs should be sufficient for this model.

Evaluate the model on the test dataset. For full marks, it is expected to achieve classification accuracy on the test set greater than 97%.

Also, please read the following [lecture](#) for an introduction to natural language processing with the Hugging Face library.

Step 2: Use the trained MobileBERT model to predict the class of the following four messages:

- You won an award! It is a free cruise trip! Call us immediately to claim it.
- Free apple phones on us for our customers, call immediately!
- I read a great book yesterday.
- March Madness has been really fun to watch this year.

Estimated running time: between 15 and 45 minutes.

Report (25 marks): (a) Report the classification accuracy for the test dataset. For full marks, it is expected to report test accuracy above 97%. (b) Report the predicted class (spam or ham) by the model for the four messages in Step 2.

Task 2: Implement adversarial text attacks against the trained MobileBERT model **using the TextAttack library**.

Step 1: Apply the TextFooler attack on the following message: Free apple phones on us for our customers, call us immediately!

Step 2: Select the first 10 messages from the test dataset, and apply the TextFooler attack to generate adversarial examples.

Step 3: Apply the WordBug attack on the first 10 messages from the test dataset to generate adversarial examples.

Step 4: Apply the PWWS (Probability Weighted Word Saliency) attack on the first 10 messages from the test dataset to generate adversarial examples.

Step 5: Describe the above three adversarial attacks. Write at least three sentences for each of the attacks to explain the methods they use for generating adversarial examples.

Step 6: Elaborate on the attacks on the MobileBERT model and the achieved success rate in the previous steps. Write your opinion on which of these attacks created adversarial messages that are the least perceptible by human users. Write between 5 to 10 sentences.

Estimated time: between 5 and 20 minutes.

Report (25 marks): (a) Report the success rate of the above attacks in Steps 1 to 4. (b) Write the analyses for Steps 5 and 6. Do not use AI assistants for writing the reports.

Part 2: Jailbreaking Attacks (50 marks)

We will use the [unsloth](#) library for finetuning LLMs. This library is built on top of Hugging Face, and applies various optimization techniques to reduce the training and inference time and resources. The notebook `Finetuning_Llama_3_8B` in the LLM Finetuning folder on Canvas provides an example code for finetuning a Llama 3.1 model with 8 billion parameters. **It is expected that your solution follows the example provided in the `Finetuning_Llama_3_8B` notebook.**

Dataset: We will use the [Alpaca](#) dataset consisting of 52,000 pairs of questions and responses for finetuning LLMs generated by OpenAI's GPT-3 model. This dataset is used for supervised finetuning of base LLMs using instruction following.

Task 1: Step 1: Train a **Qwen 2.5 LLMs using the unsloth library**. Qwen 2.5 is a family of models developed by Alibaba using a large dataset encompassing 18 trillion tokens. For this task, we will use the model with 7 billion parameters.

Follow the code in the `Finetuning_Llama_3_8B` notebook posted in the Codes folder on Canvas for training and evaluating the model.

Load the model from the folder "unsloth/Qwen2.5-7B-Instruct-bnb-4bit" in the unsloth library.

Load the Alpaca dataset from the "tatsu-lab/alpaca" folder in the Hugging Face hub.

Finetune the Qwen 2.5 7B model using the Alpaca dataset for 60 steps of parameter updates, with 32 input text sequences per step.

This task requires access to a T4 GPU which has 16 GB of RAM. If you don't use Colab, ensure that you run the code on a GPU with at least 16 GB of RAM.

If you need extra materials to learn about LLM finetuning with unsloth, please check the following tutorials:

- <https://huggingface.co/blog/mlabonne/sft-llama3>
- [https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3.1_\(8B\)-Alpaca.ipynb](https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3.1_(8B)-Alpaca.ipynb)
- [Lecture 22-LLMs](#)

Step 2: Use the trained Qwen 2.5 7B model to write a poem about Adversarial Machine Learning.

Step 3: Explain the LoRA technique for finetuning LLMs using at least 10 sentences.

Step 4: Explain the quantization technique for finetuning LLMs using at least 5 sentences.

Estimated running time: between 20 and 60 minutes.

Report (25 marks): (a) Report the training loss of the model and the generated poem from Step 2. (b) Write the explanations for Steps 3 and 4.

Task 2: Apply manually crafted adversarial prompts for jailbreaking the trained Qwen 2.5 7B model to generate harmful responses.

Step 1: Create at least 10 adversarial prompts for jailbreaking the Qwen 2.5 7B model. At least two prompts should be applied for each of the five types of jailbreaking attacks: prefix injection, refusal suppression, distractor instructions, style injection, and obfuscation scheme.

It is required that at least one of the 10 manually crafted adversarial prompts results in a harmful response by the Qwen 2.5 7B model.

It is not allowed to use the prompt examples provided in the Finetuning_Llama_3_8B notebook.

Step 2: Write a report on the success of the crafted adversarial prompts, and elaborate on the robustness of Qwen 2.5 7B to the different types of jailbreaking attacks. The report should have at least 5 sentences.

Estimated time: between 5 and 20 minutes.

Report (25 marks): (a) Report the crafted adversarial prompts and the responses by the model. (b) Write the analysis for Step 2. Do not use AI assistants for writing the reports.

Submission Documents

The assignment documents are submitted on Canvas.

1. Submit all your codes as Jupyter Notebooks. Please try to comment your codes extensively, introduce the names of all used variables, avoid one-letter variables, etc., to improve the readability of the codes. For instance, for this assignment you can submit two Jupyter Notebooks with the codes. You don't need to submit the dataset on Canvas.
2. Prepare a brief report with tables, graphs, and results: the report can be prepared either as a separate MS Word/PDF file, or it can be integrated into your Jupyter Notebooks by typing your analysis directly into text cells in the Jupyter Notebooks.